

Learning Material: CMU 11-785

Introduction to Deep Learning

Lecture 19 Transformer

Presented by Yalei Zhao

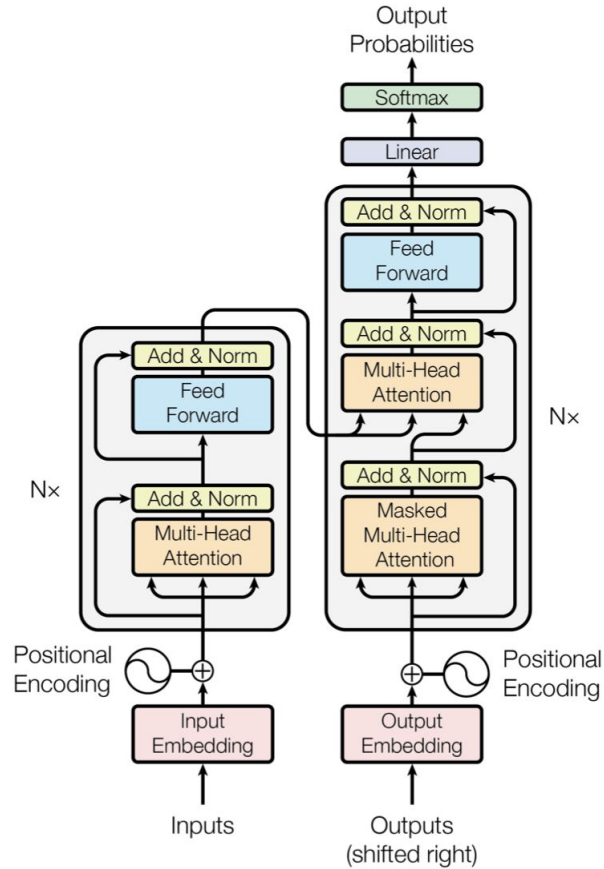
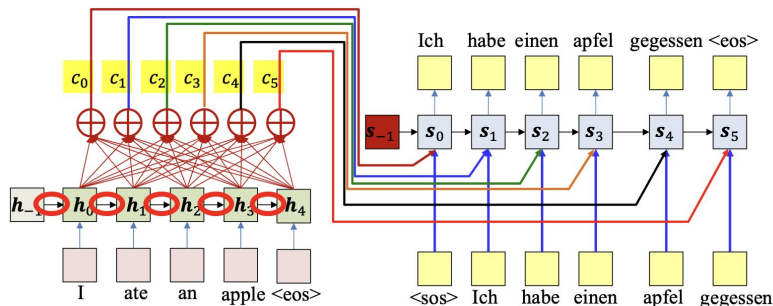


Figure 1: The Transformer - model architecture.

- Review:
 - Recurrence to Self attention (Q,K,V)
 - multihead self attention
- Transformer Architecture
 - Tokenization & Embedding
 - 3 types of Position Encoding
 - Absolute (original)
 - Relative
 - Rotary
- 3 types of Attention
 - Attention in Encoder: bidirectional
 - Attention in Decoder: Masked
 - Cross Attention
- 3 types of Transformer
 - Encoder - Decoder (original)
 - Encoder only (BERT)
 - Decoder only (generative)
- Efficiency attention:
 - Math
 - Execution

Recurrence to self attention

Recap: Attention Models



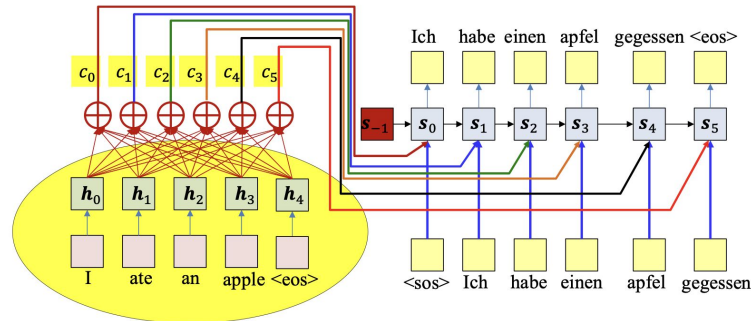
- Problem: Because of the recurrence, the hidden representation for any word is also influenced by *all* preceding words
 - The decoder is actually paying attention to the sequence, and not just the word
- If the decoder is automatically figuring out which words of the input to attend to at each time, is recurrence in the input even necessary?

102

Problem with recurrence:

- slow
- long-range info weakens
- “pass the word”

Recap: Non-recurrent encoder



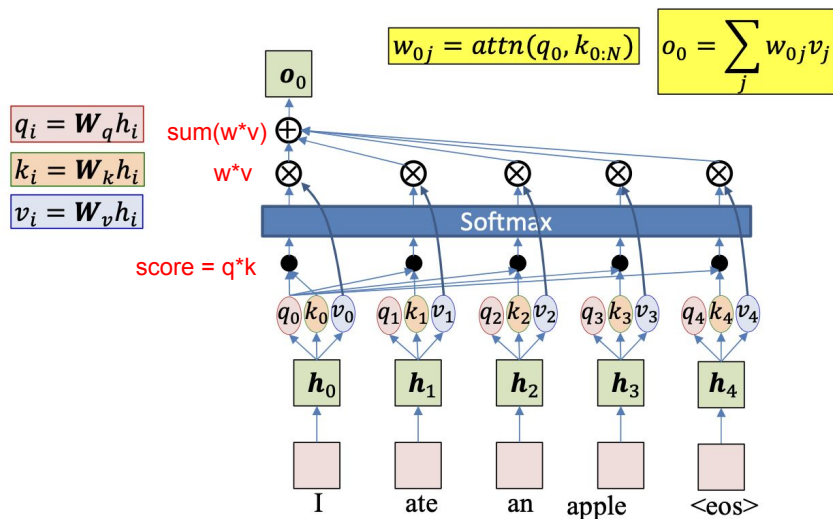
- The encoder in a sequence-to-sequence model can be composed without recurrence.
- Use the attention framework itself to introduce context-specificity in embeddings
 - “Self” attention

105

Benefits of self-attention:

- parallel, faster
- captures long-range context
- “direct communicate”

Self attention (in encoder)



W (softmaxed attention weights): match between **Query** and **Key**
– determines how much attention to give

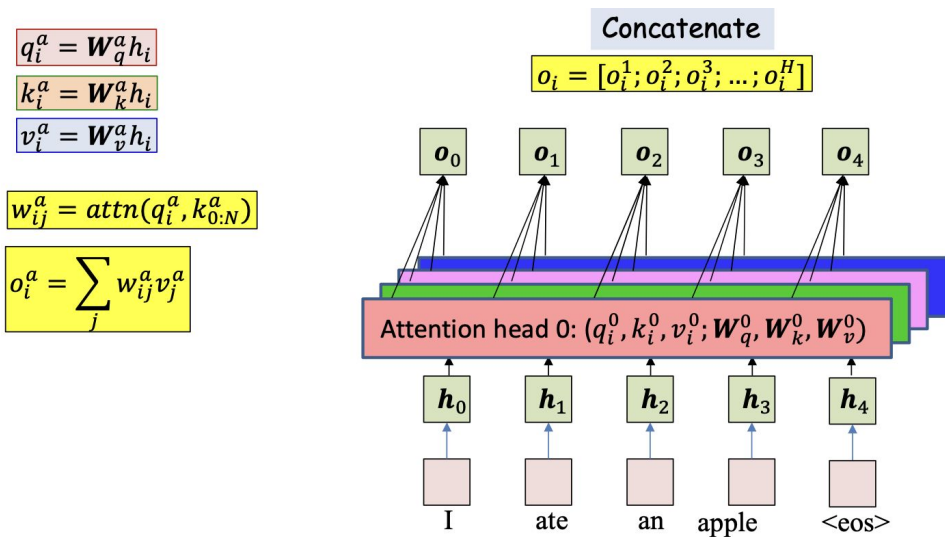
V (value): actual information passed forward
– what gets combined into the output

Example:

- Query("ate") - intend to find food character
- Key("apple") - high matching
- Value("apple") - apple has more info than food, e.x. *noun, context meaning, relations*

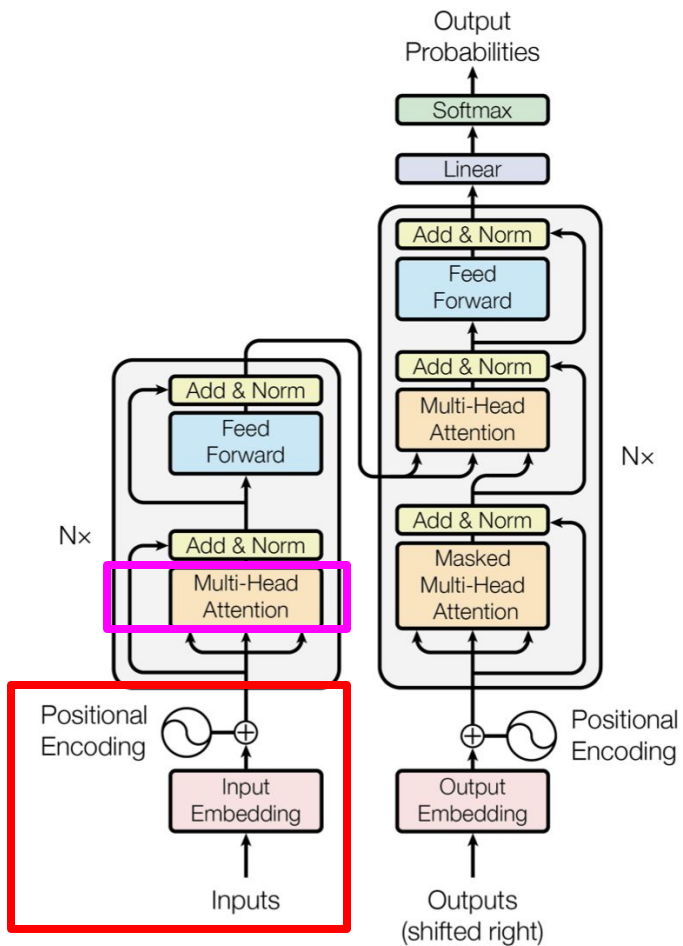
- Compute query-key-value sets for every word
- For each word
 - Using the query for that word, compute attention weights for all words using their keys
 - Compute updated representation for the word as attention-weighted sum of values of all words

Multi-head self attention (in encoder)



- Single-head self attention:
 - one view of the sentence
 - captures one type of relationship
- Multi-head attention:
 - multiple views in parallel
 - each head focuses on different aspects: *e.x. syntax (grammar), semantics, position*

- We can have *multiple* such attention “heads”
 - Each will have an independent set of queries, keys and values
 - Each will obtain an independent set of attention weights
 - Potentially focusing on a different aspect of the input than other heads
 - Each computes an independent output
- The final output is the concatenation of the outputs of these attention heads
- “MULTI-HEAD ATTENTION” (actually Multi-head *self* attention)



So far: Multi-Head Attention in encoder

Next: Input Embedding & Position Encoding

Figure 1: The Transformer - model architecture.

Word Tokenization & Embedding

- Tokenization: Maps a word to one/multiple tokens.
E.x. “I ate an apple.” — [“I”, “ate”, “an”, “apple”, “.”]
- Token Index: Maps token to indices.
E.x. [“I”, “ate”, “an”, “apple”, “.”] — [40, 28397, 448, 30366, 13]
- Embedding: Maps discrete token index to a continuous vector.

E.x.

Token Index	Token Embedding
0	[0.235, -1.256, 3.513, ..., -0.187]
1	[1.291, -2.012, 0.624, ..., -1.291]
2	[0.536, 0.012, -0.024, ..., 2.345]
...	...
Vocab Size V	[0.131, 2.102, 0.935, ..., -0.125]

Position encoding

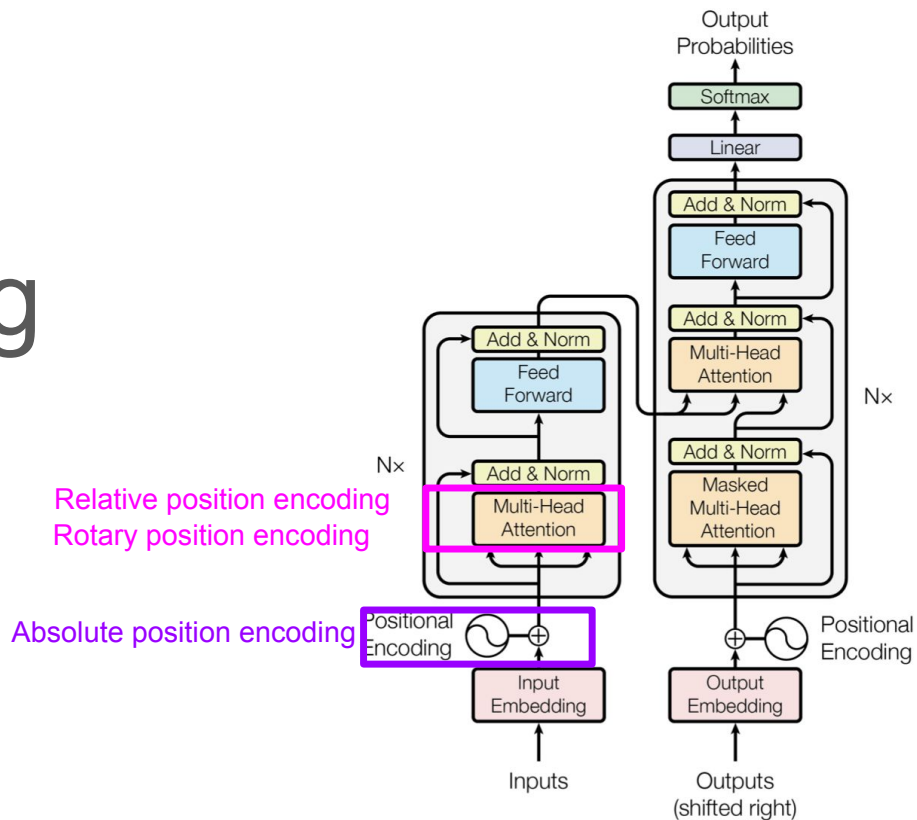


Figure 1: The Transformer - model architecture.

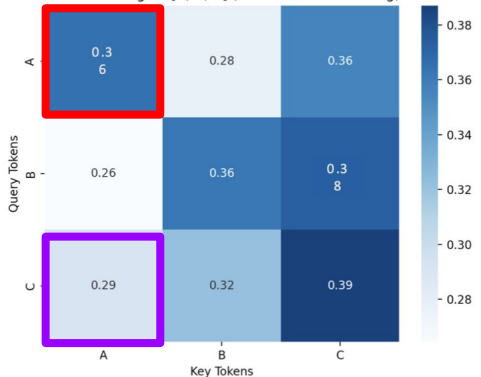
Position Encoding

- Why do we need them?
 - Self-attention is permutation-invariant!
- Considering a sequence of
 - [A, B, C] vs. [C, A, B]
- No position information!

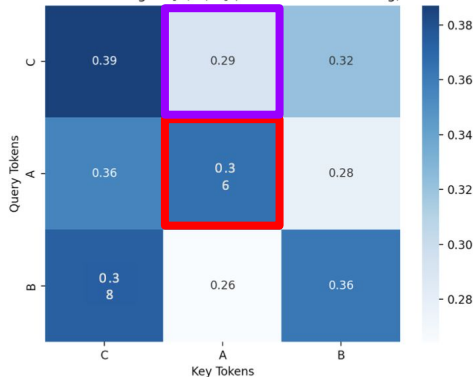
Self-attention treats input as a **set**, not a **sequence**. BUT we want to preserve the **order**.

E.x. “I ate apple.” vs. “apple ate I.”

Attention Weights [A, B, C] (No Positional Encoding)



Attention Weights [C, A, B] (No Positional Encoding)



A-A

C-A

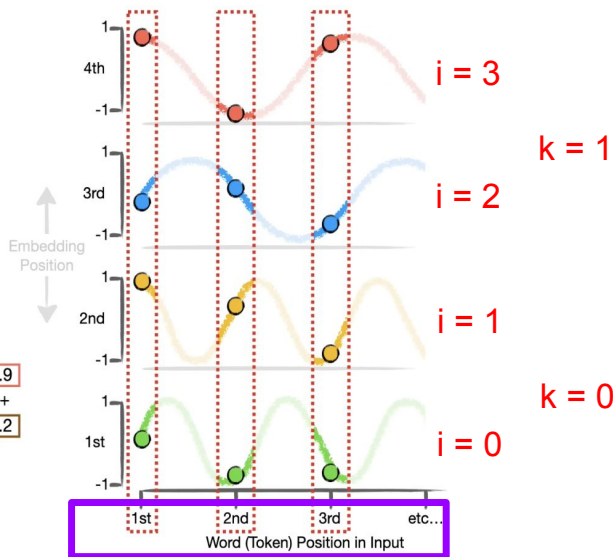
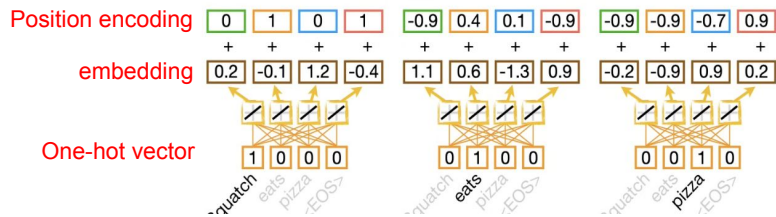
Position encoding (absolute)

$$\dim(\text{Position encoding}) = \dim(\text{embedding})$$



Thus, each input word ends up with a unique sequence of position values.

$$f(t)^{(i)} := \begin{cases} \sin(\omega_k \cdot t), & \text{if } i = 2k \\ \cos(\omega_k \cdot t), & \text{if } i = 2k + 1 \end{cases}$$



Absolute position encoding

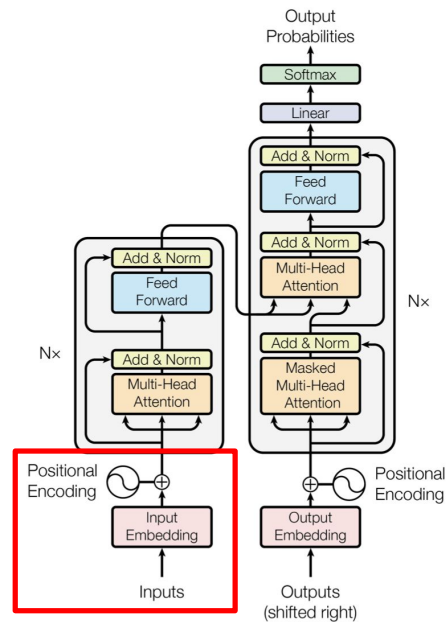


Figure 1: The Transformer - model architecture.

Absolute position encoding

- Fixed embedding according to position index
- Add position encoding to word embedding
- Example:

"I ate an apple"

"Yesterday I ate an apple"

"apple" is at position: 4 in A; 5 in B

Cons:

- sensitive to sentence length shifts
- doesn't explicitly model distance between words
- weaker generalization to longer sequences

Relative position encoding

- Relative distances between token
- Add directly to attention score: $QK^T + \text{bias}(i-j)$
- Example:

"I ate an apple"

[-3, -2, -1, 0]

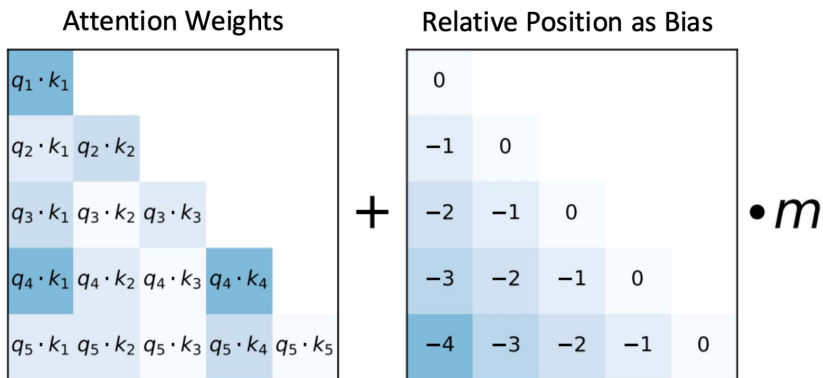
"Yesterday I ate an apple"

[-4, -3, -2, -1, 0]

Pro:

Relative encoding keeps relationships between words unchanged, even when sentence length or structure shifts.

Relative position encoding



- Relative distance as offset added to attention matrix
- Absolute position embedding not needed

m amplifies the importance of distance

If $m > 0$:

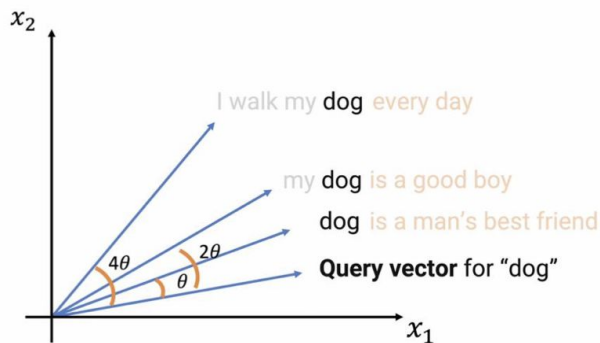
- Closer tokens,
less penalty,
higher score,
smaller probability after softmax
- Far tokens,

more penalty,
lower score,
smaller probability after softmax

Rotary position encoding

Rotate Q and K before computing attention score (QK^T)

Simpler than relative position encoding, no extra bias matrix needed



- Rotate the embedding in 2D space

$$\langle f_q(x_m, m), f_k(x_n, n) \rangle = g(x_m, x_n, m - n)$$

$$f_q(x_m, m) = (W_q x_m) e^{im\theta}$$

$$f_k(x_n, n) = (W_k x_n) e^{in\theta}$$

$$g(x_m, x_n, m - n) = \text{Re} \left[(W_q x_m) (W_k x_n)^* e^{i(m-n)\theta} \right]$$

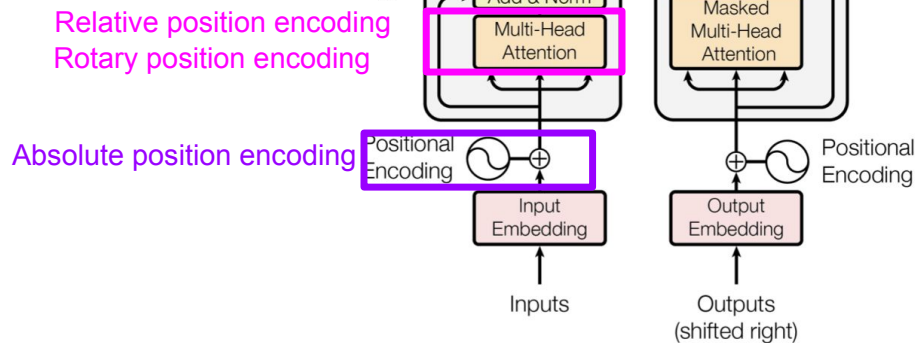


Figure 1: The Transformer - model architecture.

Feed Forward

- Just a MLP!

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

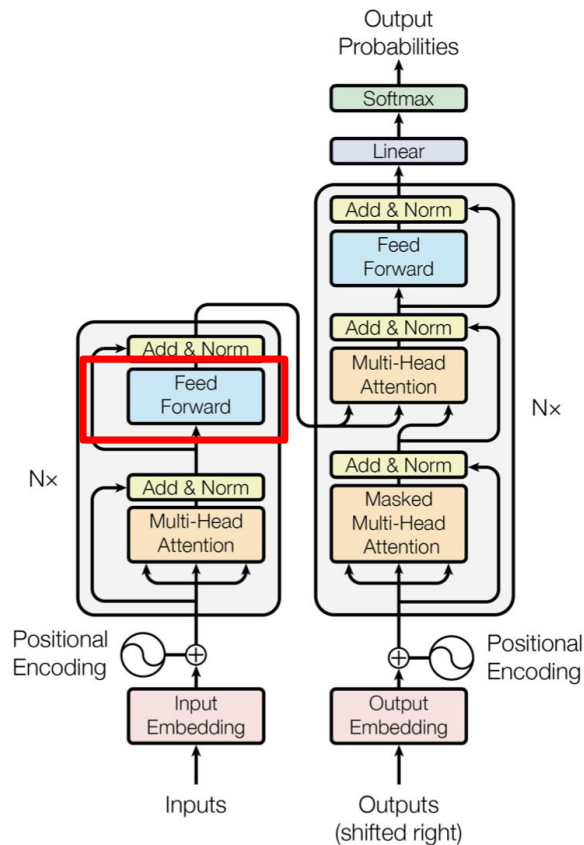
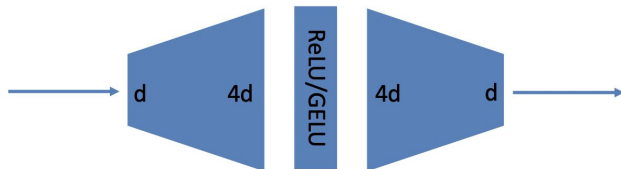


Figure 1: The Transformer - model architecture.

Add & Norm

Residual and Normalization

- Each layer in Transformer has:
 - A residual connection
 - A normalization layer
- Layer Norm. normalize each token by its embedding size dimension
 - For more stable training

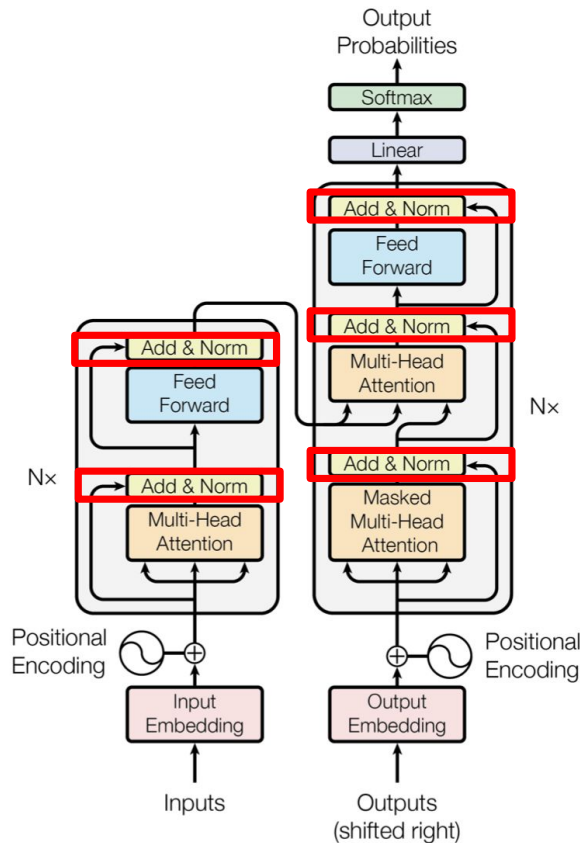
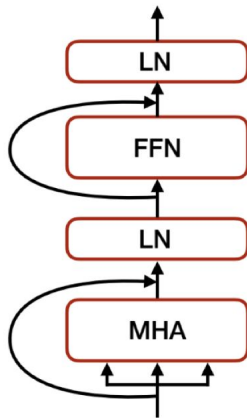
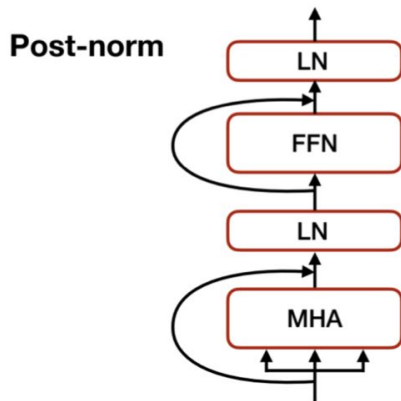


Figure 1: The Transformer - model architecture.

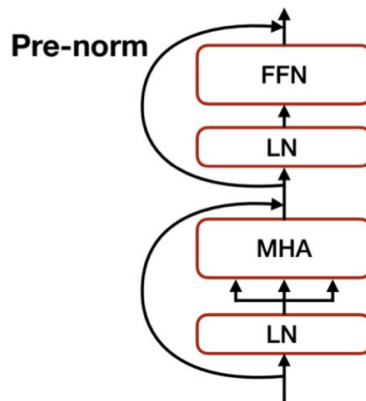
Position of Normalization

- Post-Norm vs Pre-Norm

$$\mathbf{x}_{t+1} = \text{Norm}(\mathbf{x}_t + F_t(\mathbf{x}_t))$$



$$\mathbf{x}_{t+1} = \mathbf{x}_t + F_t(\text{Norm}(\mathbf{x}_t))$$



- Pre-Norm is easier and more stable to train
- Post-Norm tends to present better performance if properly trained

3 Types of Attention

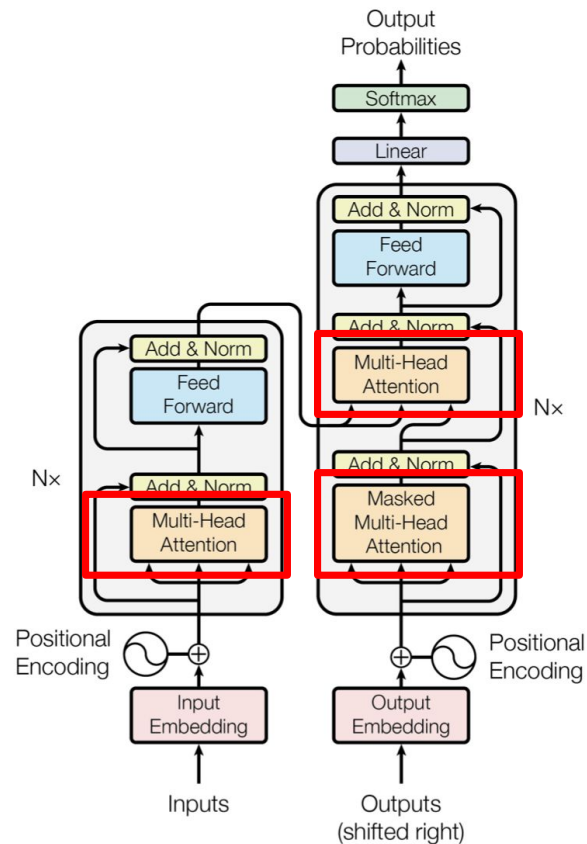


Figure 1: The Transformer - model architecture.

Encoder Self Attention

For a query from encoder,
it can attend to any key from
the encoder

Understand input

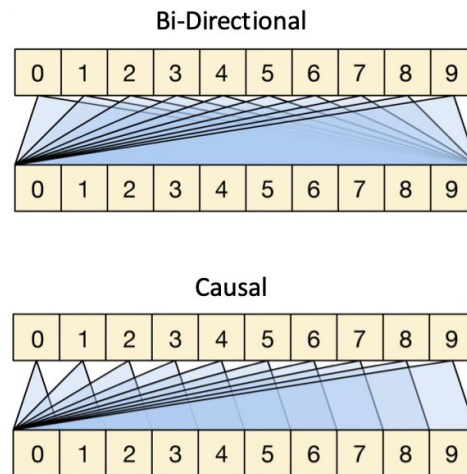
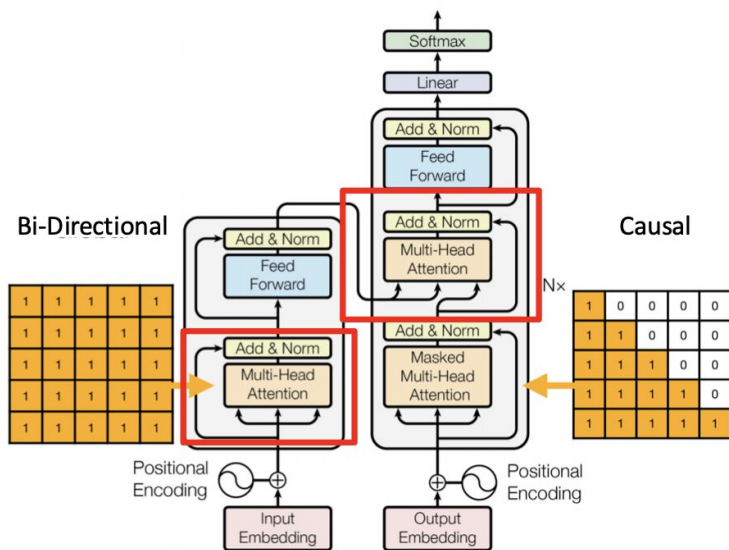
Decoder Self Attention

For a query from decoder at
position t, it can only attend to
before position t keys (future
masked)

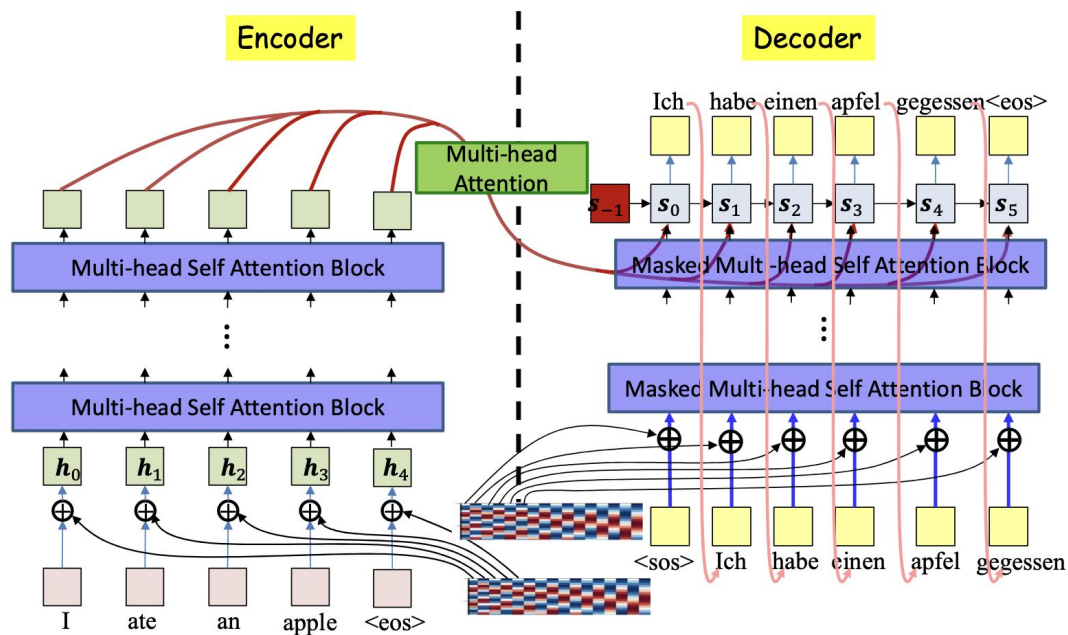
Understand what has been
generated so far

Cross Attention

For a query from decoder,
it can attend to any key
from the encoder



Transformer training vs inference



Training:

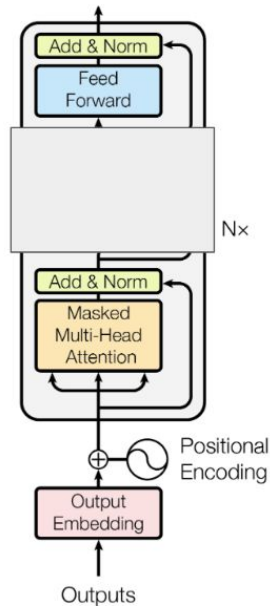
At each step t :

- Use true tokens up to $t-1$ as decoder input
- Form a query from decoder state at t – attend to encoder outputs – produce prediction for token t
- Compare prediction with true token at t – compute loss
- Repeat: always use true previous tokens, not model predictions

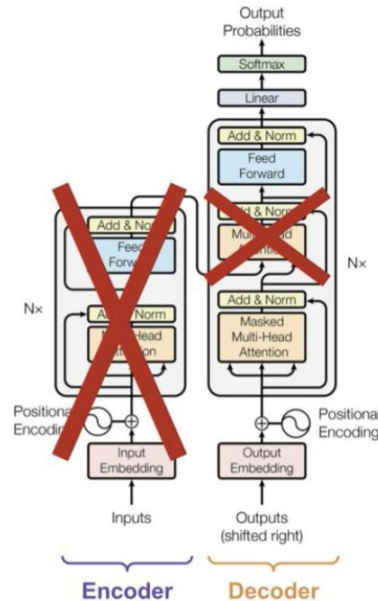
Inference:

- Use the predicted token at position t as the input for step $t+1$.

3 Types of Transformers



Encoder only



Decoder only

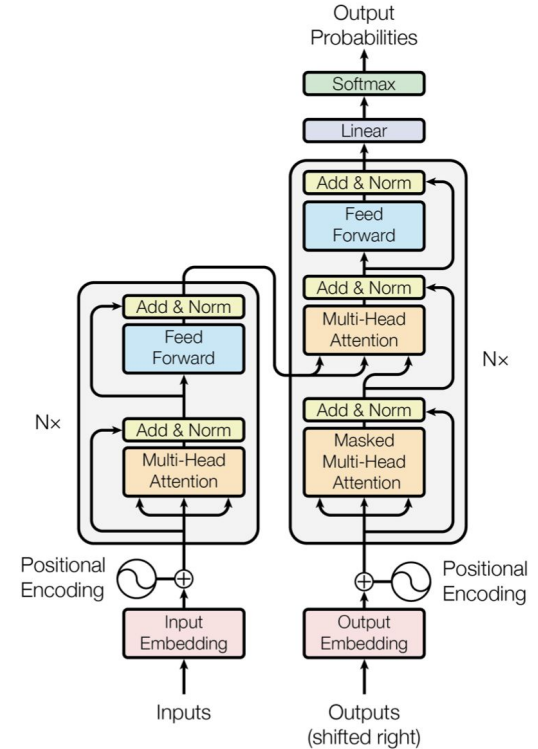


Figure 1: The Transformer - model architecture.

Encoder - Decoder

Encoder - Decoder

Transform: Learn mapping between two sequences

E.x.

Input: "I ate an apple"

Output: "J'ai mangé une pomme"

How it works

- Encoder reads full sentence
- Decoder generates new sentence step-by-step
- Uses **cross-attention** to connect input & output

Intuition: Read first, then write something different"

Decoder Only

Generate: Learn how text continues

E.x.

Input: "I ate an apple"

Output: "I ate an apple and it was delicious"

How it works

- Single sequence
- Uses **causal self-attention**
- Predict next word

Intuition: "Keep writing the story"

Encoder Only (BERT)

Understand: Learn what text mean

E.x.

Input: "I ate an apple"

Output: representation / understanding

How it works

- Sees full sentence
- Uses **bidirectional self-attention**
- No generation

Intuition: "Read and understand"

Decoder-only Transformer

Why we want to have a decoder-only transformer?

- Original Attention is all you need paper: encoder-decoder transformer
- Cons:
 - more parameters
 - Slower
 - More complex training
 - Unnecessary for simple tasks

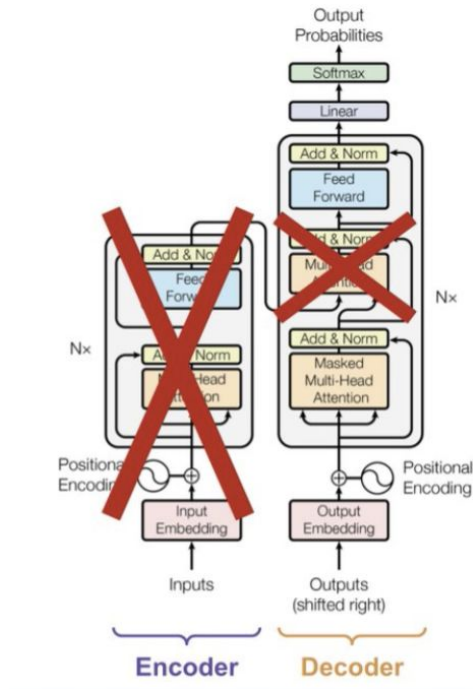
Instead of:

Encoder: "I ate an apple"

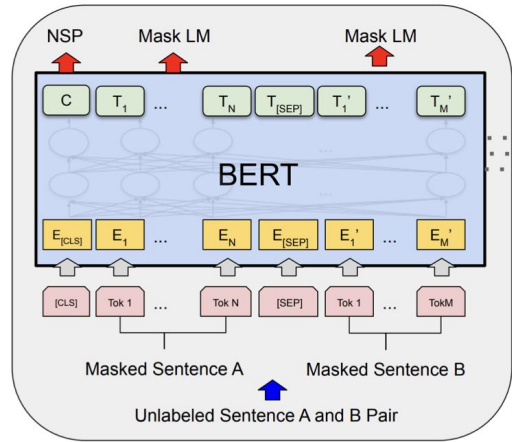
Decoder: "J'ai mangé une pomme"

GPT does:

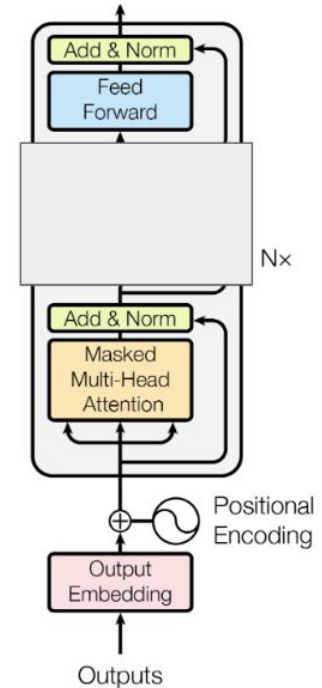
"Translate English to French: I ate an apple ..."



Encoder-only Transformer (BERT)



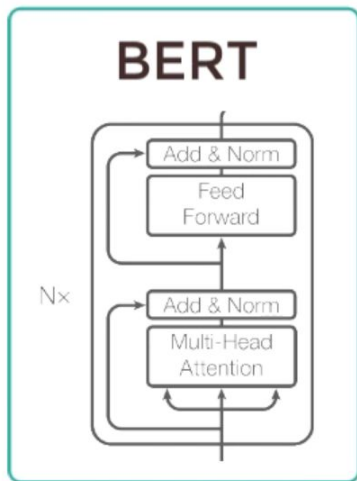
- Some tokens are masked
- Input is complete in position, just partially hidden in content
- Use both left and right context to predict the masked



E.x. Amazon chatbox – “My package was not delivered.”

BERT understand the info “not delivered” , and a later decision layer: “Refund”

Bidirectional Encoder Representation from Transformers



Problems to Solve

- Neural Machine Translation
- Question Answering
- Sentiment Analysis
- Text summarization

Needs Language understanding

How to solve Problems

- Pretrain BERT to understand language
- Fine tune BERT to learn specific task

Efficient Attention Mechanism

THE ISSUE: Quadratic Complexity

L : sequence length
 d : embedding dimension

- Self-attention has quadratic complexity to input length

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

– $O(L^2d)$ FLOPS

- Many attempts for reducing the quadratic complexity to linear

- Linear Attention
- Flash Attention
- Grouped Query Attention
- Multi Query Attention
- Multi-head Latent Attention

Computing QK^T :

$$QK^T : (L \times d)(d \times L) = L \times L$$

- Each entry = dot product of length d
- Number of entries = L^2
- $O(L^2d)$

Softmax:

- $O(L)$ per row, softmax $O(L^2)$

V :

$$(L \times L)(L \times d) = L \times d$$

- $O(Ld)$ per row, L rows give $O(L^2d)$

61

$$O(L^2d) + O(L^2) + O(L^2d) \approx O(L^2d)$$

Linear attention (different math, reduce computation)

- Modification on Softmax

$$\text{Softmax}(QK^T)V = \frac{\exp(QK^T)}{\sum_{i=1}^L \exp(QK_i^T)}V \xrightarrow{\phi(Q) \cdot \phi(K)^T} \frac{\text{sim}(Q, K)}{\sum_{i=1}^L \text{sim}(Q, K_i)}V$$

*for a fixed query
look at all keys*

- Kernel function

$$\text{sim}(Q, K) = \cancel{\phi(Q)} \cdot \cancel{\phi(K)} = \phi(Q)\phi(K)^T$$

$L \times d'$ $d' \times L$

- Linear form of attention

$$\frac{\boxed{\phi(Q)\phi(K)^T}}{\sum_{i=1}^L \phi(Q)\phi(K_i)^T}V = \frac{\phi(Q) \boxed{\phi(K)^TV}}{\phi(Q) \sum_{i=1}^L \phi(K_i)^T}$$

$L \times d'$ $d' \times L$ $L \times d$

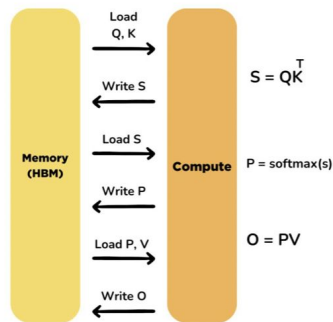
association property

$O(L^2)$

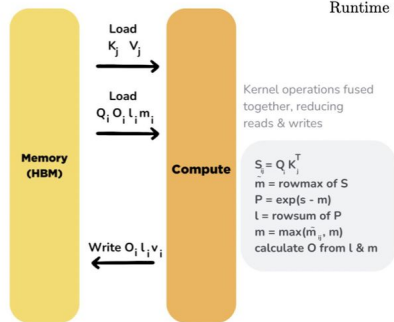
$O(\underbrace{d'd})$
embedding dim

Flash attention (reduce memory access)

Standard Attention Implementation

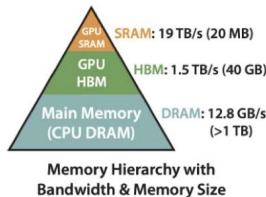


Flash Attention



Initialize O, l and m matrices with zeroes. m and l are used to calculate cumulative softmax. Divide Q, K, V into blocks (due to SRAM's memory limits) and iterate over them, for i is row & j is column.

Attention	Standard	FLASHATTENTION
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3



SRAM: Static Random-Access Memory

on chip, fast but low memory

HBM: Host Memory Buffer

global, slower but larger moeory

Standard Attention:

“Write everything down, then process later”

Flash Attention:

“Process immediately, don’t write it down”

not write out Q, K, V one by one then perform softmax, directly compute to end after softmax result

Flash Attention

Without Tiling 32 access

A					B					C			
1	2	0	5	X	3	1	6	3	=	C1,1	C1,2	C1,3	C1,4
4	8	2	-1		-7	5	0	8		C2,1	C2,2	C2,3	C2,4
3	1	6	3		1	2	0	5		C3,1	C3,2	C3,3	C3,4
-7	5	0	8		0	3	5	1		C4,1	C4,2	C4,3	C4,4

$$C_{1,1} = 1 \times 3 + 2 \times (-7) + 0 \times 0 + 5 \times 0 = -11$$

$$C_{1,2} = 1 \times 1 + 2 \times 5 + 3 \times 9 + 4 \times 13$$

$$C_{1,3} = 1 \times 2 + 2 \times 6 + 3 \times 10 + 4 \times 14$$

$$C_{2,1} = 5 \times 1 + 6 \times 5 + 7 \times 9 + 8 \times 13$$

$$C_{2,2} = 5 \times 2 + 6 \times 6 + 7 \times 10 + 8 \times 14$$

.....
.....

With Tiling 16 access

A					B					C	
1	2	0	5	X	3	1	6	3	=	C1,1	C1,2
4	8	2	-1		-7	5	0	8		C2,1	C2,2
3	1	6	3		1	2	0	5			
-7	5	0	8		0	3	5	1			

$$C_{11} = A_{11} \times B_{11} + A_{12} \times B_{21}$$

$$C_{12} = A_{11} \times B_{12} + A_{12} \times B_{22}$$

$$C_{21} = A_{21} \times B_{11} + A_{22} \times B_{21}$$

$$C_{22} = A_{21} \times B_{12} + A_{22} \times B_{22}$$

Dao, Tri, et al. "Flashattention: Fast and memory-efficient exact attention with io-awareness." Advances in Neural Information Processing Systems 35 (2022): 16344-16359.

How FlashAttention Accelerates Generative AI Revolution: <https://www.youtube.com/watch?v=BMAC117au4A>

Flash Attention

Softmax

Computation requires two loops: one to calculate the normalizing factor (the sum of exponentials) and another to compute the attention weights by dividing each exponentiated value by this factor.

$$\frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$

Safe Softmax

Requires three loops: one to find the maximum value (for numerical stability), one to compute the normalizing factor, and one to obtain the attention weights.

for $i \leftarrow 1, N$ do

$$m_i \leftarrow \max(m_{i-1}, x_i)$$

for $i \leftarrow 1, N$ do

$$d_i \leftarrow d_{i-1} + e^{x_i - m_N}$$

for $i \leftarrow 1, N$ do

$$a_i \leftarrow \frac{e^{x_i - m_N}}{d_N}$$

Online Softmax

Requires two loops: one to find the maximum value (and to compute the normalizing factor, and one to obtain the attention weights.

for $i \leftarrow 1, N$ do

$$m_i \leftarrow \max(m_{i-1}, x_i)$$

$$d'_i \leftarrow d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

for $i \leftarrow 1, N$ do

$$a_i \leftarrow \frac{e^{x_i - m_N}}{d'_N}$$

Suppose $Q: L \times D$ is 3×3

$X = [2, 5, 3]$ (one row of QK^T) $V[1] = [1, 0, 0], V[2] = [0, 2, 0], V[3] = [0, 0, 3]$

Flash Attention

$$O = \sum_i \text{Softmax}(x_i) \cdot V$$

$$O = \left[\frac{1 \cdot e^2}{e^2 + e^5 + e^3} \quad \frac{2 \cdot e^5}{e^2 + e^5 + e^3} \quad \frac{3 \cdot e^3}{e^2 + e^5 + e^3} \right]$$

Fused computation to one loop!

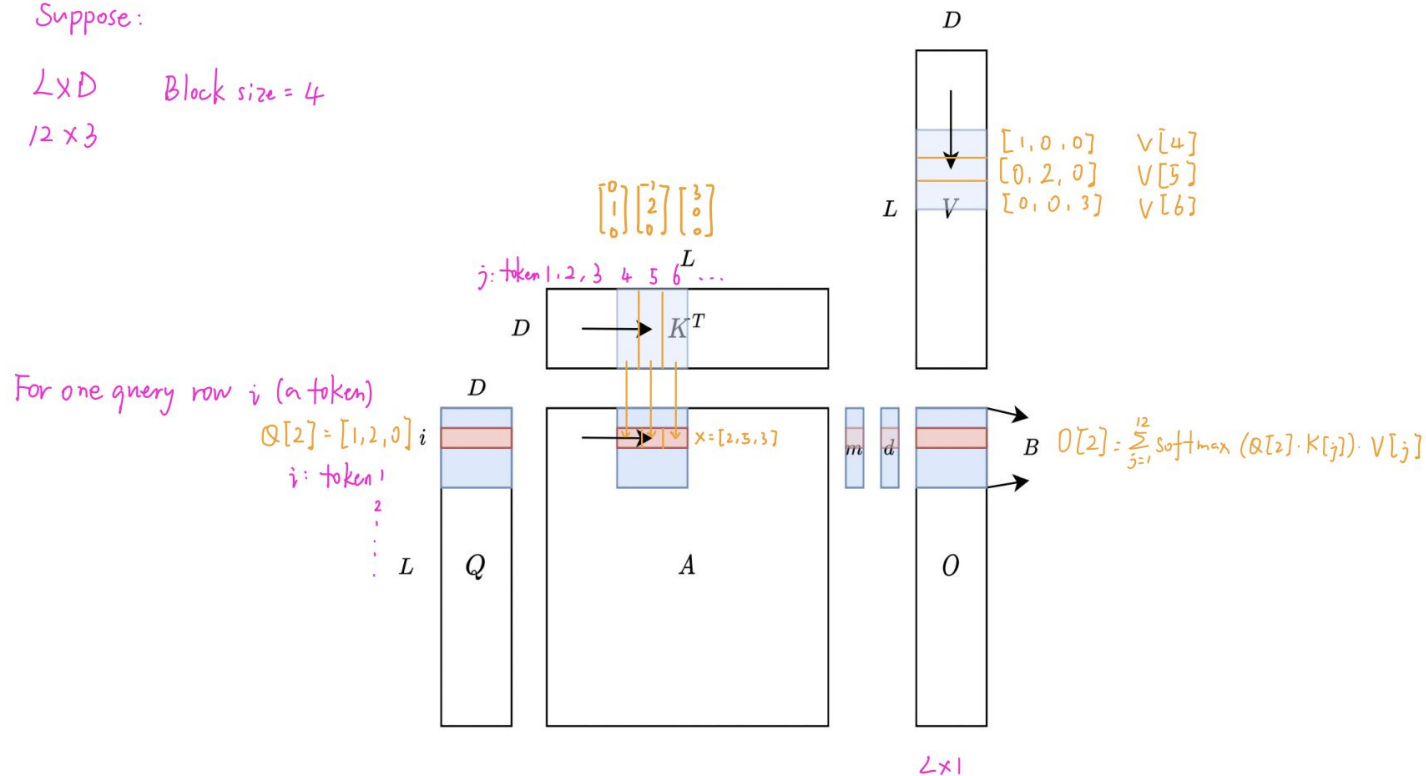
$$\begin{aligned}
 & i=1 \quad \quad \quad i=2 \quad \quad \quad i=3 \\
 & \text{for } i \leftarrow 1, N \text{ do} \\
 & \quad x_1 = 2 \quad \quad x_2 = 5 \quad \quad x_3 = 3 \quad m_3 = \max(m_2, x_3) = 5 \\
 & \quad m_1 = 2 \quad \quad m_2 = 5 \\
 & \quad d'_1 = e^{2-2} = 1 \quad d'_2 = 1 \cdot e^{2-5} + 1 = e^{-3} + 1 \quad d'_3 = (e^{-3} + 1) \cdot e^{5-5} + e^{3-5} = e^{-3} + 1 + e^{-2} \\
 & \quad o'_1 = \frac{e^{2-2}}{1} \cdot V[1] \quad o'_2 = o'_1 \cdot \frac{d'_1 e^{2-5}}{d'_2} + \frac{e^{5-5}}{d'_2} V[2] \quad o'_3 = o'_2 \frac{d'_2 e^{5-5}}{d'_3} + \frac{e^{3-5}}{d'_3} V[3] \\
 & \quad = [1 \ 0 \ 0] \quad \quad \quad o'_i \leftarrow o'_{i-1} \frac{d'_{i-1} e^{m_{i-1}-m_i}}{d'_i} + \frac{e^{x_i-m_i}}{d'_i} V[i, :] \\
 & \quad \text{end} \quad \quad \quad = [1 \ 0 \ 0] \frac{e^{-3}}{e^{-3}+1} + \frac{1}{e^{-3}+1} [0 \ 2 \ 0] \\
 & \quad \quad \quad = \left[\frac{e^{-3}}{e^{-3}+1} \quad \frac{2}{e^{-3}+1} \quad 0 \right] \\
 & \quad \quad \quad O[k, :] \leftarrow o'_N + \frac{e^{-2}}{e^{-3}+1+e^{-2}} [0 \ 0 \ 3] \\
 & \quad \quad \quad = \left[\frac{e^{-3}}{e^{-3}+1+e^{-2}} \quad \frac{2}{e^{-3}+1+e^{-2}} \quad \frac{3e^{-2}}{e^{-3}+1+e^{-2}} \right] \\
 & \quad \quad \quad = \left[\frac{1 \cdot e^2}{e^2 + e^5 + e^3} \quad \frac{2 \cdot e^5}{e^2 + e^5 + e^3} \quad \frac{3 \cdot e^3}{e^2 + e^5 + e^3} \right]
 \end{aligned}$$

Flash Attention

Suppose:

$L \times D$ Block size = 4

12×3



Dao, Tri, et al. "Flashattention: Fast and memory-efficient exact attention with io-awareness." Advances in Neural Information Processing Systems 35 (2022): 16344-16359.

<https://courses.cs.washington.edu/courses/cse599m/23sp/notes/flashattn.pdf>